

How the answers were built

Findings · architecture and decisions · EDA and data quality · semantic model reference · security and production readiness · AI and advanced analytics · explainer briefs and Q&A

Amr Dawoud

BI Consultant & AI Engineer · 26 September 2026

0. Findings and recommendations

All figures are recomputed from the governed gold layer (scripts/findings.py → docs/findings_numbers.json) and match the semantic-model measures named below. Unless stated otherwise, the period is Jan 2025 – Aug 2026, and year-on-year is Jan–Aug 2026 vs Jan–Aug 2025.

Headline

Net revenue grew **+7.6%** (7.55M → 8.13M EGP) and volume **+10.6%**, but gross profit fell **–5.4%** (2.15M → 2.03M). Gross margin dropped from **28.4% to 25.0%** as a clean step in January 2026. The bridge has two parts: - **Mix: –1.50 pp**. Milk, the lowest-margin category, grew +109% and rose from 23% to 46% of revenue. - **Cost: –1.94 pp**. Five Milk SKUs were costed at 6% above their ERP standard.

Nothing else moved: margins in Yogurt, Juice and Cheese are unchanged. Growth is real volume, but it is growth in the least profitable products, costed on the wrong basis.

#	Finding	Evidence	Business impact	Recommendation	How to measure success
1	Standard costs are stale. Milk SKUs 1001–1005 have cost exactly 1.06× the ERP standard since Jan 2026; list prices have not changed.	COGS ÷ (Qty × Standard Cost) = 1.060 on 14,544 lines, 1,000 everywhere else (DQ rule SAL-04). Milk margin is 23.3% at standard vs 19.0% actual.	158k EGP of cost above standard in 8 months (≈ 237k annualised). Pricing and mix decisions based on standard margin overstate Milk profitability by 4.3 pp.	Refresh Milk standard costs now and move to quarterly re-costing. Review Milk pricing at the true 19% margin.	<i>Cost Variance vs Standard</i> → 0; <i>Gross Margin % at Standard Cost vs Gross Margin %</i> gap closes; Milk GM % by month.
2	Plan attainment hides a quality gap. Plants hit 99.8% of planned units, but only 95.3% are good.	Scrap: Plant 1 A/B 1.8%, Plant 1 C 5.4%, Plant 2 A/B 5.5%, Plant 2 C 9.0% . Downtime is 164 vs 107 min/order at Plant 2. Overtime per head and scrap correlate across plant × shift cells (r = 0.89). Overtime is structural (P2 C ≈ 36 h/person/month) and flat year on year , contrary to the brief.	298k EGP avoidable scrap at the Plant 1 average rate (394k against the Plant 1 day-shift stretch benchmark). Plant 2 over-produces (102.6%) to compensate for scrap.	Standard work and changeover and QA practice from Plant 1 A/B rolled out to Plant 2. Rebalance Shift C staffing and order loading. Target overtime reduction at Plant 2 C.	<i>Good Output Attainment %</i> , <i>Scrap Rate %</i> by plant × shift, <i>Avoidable Scrap Cost</i> , <i>Overtime Hours per Employee</i> .
3	Nile Distribution (Cairo) is stuffing its channel. It is the only distributor whose sell-in and sell-out diverge.	2026 YTD: sell-in 41.1k units vs sell-out 23.8k (sell-through 58% vs ≈ 88% for peers). Stock rose from ≈ 2.5k to 7.5–8.9k units; months of cover 0.6 → 2.75. ERP shows Nile <i>growing</i> +5.2%.	198k EGP of stock above normal cover sitting in Nile's warehouse, i.e. revenue pulled forward with return/write-back risk. Cairo also shows the lowest target attainment index (0.65).	Cap Nile allocation at sell-out plus 0.6 months of cover. Move distributor incentives and targets from sell-in to sell-out. Run a joint stock audit.	<i>Sell-through %</i> ≥ 85%, <i>Distributor Months of Cover</i> ≈ 0.6, <i>Excess Channel Stock Value</i> → 0.
4	Stock was never re-allocated when demand moved , so Milk runs lean while Yogurt, Juice and Cheese age. The root cause is a biased forecast copied straight into production.	Warehouse stock stays at ≈ 545 units per SKU per DC in both years. 2026 cover: Milk 41 days (shelf life 14), Yogurt 116 (28), Juice 109 (60), Cheese 111 (90). MRP forecast bias: Yogurt +19%, Milk –8%. Plan = forecast – 1 unit (r = 0.99999). Expired Milk returns: 62 in all of 2025 vs 100 in Jan–Aug 2026 alone.	549k EGP of stock with ≤ 25% of shelf life left at the latest snapshot (28% of stock value; 470k in an average week). <i>Caveat</i> : warehouse quantities look to be on a larger scale than invoices, so treat values as relative.	Deploy the bias-corrected forecast (Section 4). Set DC cover targets by shelf life. Rebalance Yogurt/Cheese/Juice stock toward demand. Capture order lines to measure real stockouts.	<i>Forecast Bias %</i> → 0 ± 3%, <i>Near-Expiry Stock %</i> , <i>Days of Cover vs shelf life</i> , <i>Stockout Proxy %</i> (until fill rate exists).

#	Finding	Evidence	Business impact	Recommendation	How to measure success
5	Promotions can be measured, and they do not pay back.	Invoice lines linked to promotions by product + region + date, against an 8-week pre-promotion baseline. 0 of 37 measurable promotions cover their support cost; average lift -12%. Stated discount 9.5% but only 3.6% appears on invoices (the same as non-promoted lines). The exception, PRM-9999 (25% deep discount, Juice, Cairo, 120k support), is the only promotion whose discount hit invoices, and it had 0% lift , losing 122k.	1.87M EGP of support (36% of gross profit) with a net return of -1.87M. Promotions touch < 1% of revenue.	Pause untargeted trade discounts. Require a PromotionID on invoices and a test/control design before funding. Stop deep discounts on non-elastic SKUs.	<i>Promotion ROI %, Promotions Paying Back, Average Volume Lift %, Discount Applied on Invoice % vs stated.</i>

Three issues management did not ask about

1. **The target file is unusable.** Targets total 459.5M against 19.2M actual ($\approx 24\times$), with every target implying exactly 35 EGP per unit and a regional split (Cairo 2.3 $\times$ Canal) that does not match sales. Attainment is $\approx 4\%$ everywhere. Commercial performance management currently has no valid yardstick. *Action:* Finance restates targets; the index is used meanwhile.
2. **The CRM pipeline is not being managed.** 91.9M of "open" pipeline, 91% of which is past its expected close date. Activity volume has no relationship with account revenue ($r = 0.10$), and CRM covers only Modern Trade and Key Accounts (24% of revenue). *Action:* pipeline hygiene rules, CRM coverage for Traditional Trade and Wholesale, and activity targets tied to outcomes.
3. **Master data and payroll controls.** There are 8 duplicate customer records carrying 819 invoices, 185 invoices with free-text regions, and territory codes shared across regions. **28 employees were paid before their hire date (112 employee-months).** *Action:* a customer MDM owner, and an HR/Finance payroll control review.

1. Architecture and technical decisions

1.1 What was built (live in Microsoft Fabric)

Layer	Fabric item(s)	What it does
Landing	lh_bronze/Files/landing/<source>/load_date=YYYY-MM-DD/	Immutable drop zone for the five source systems (CSV "SQL extracts", JSONL "API feeds", CSV master files). One folder per delivery = replayable history.
Bronze	lh_bronze + nb_01_bronze_ingest	Metadata-driven ingestion from config/ingestion_config.json (22 entities: source, format, table, load type, business key, sensitivity, owner). Schema-on-read (all strings), audit columns (_source_file, _load_date, _batch_id, _ingested_at), append-only, _ingestion_log makes re-runs idempotent.
Silver	lh_silver + nb_02_silver_transform	Typed, conformed, de-duplicated. Region vocabulary conformed (9 labels → 6), duplicate customers mapped to a master, CRM float/blank IDs repaired, distributor SKUs mapped to ERP products. Sales loads incrementally (watermark + Delta MERGE). 17 DQ rules write to dq_issues / dq_run_history; rules that make numbers <i>wrong</i> stop the pipeline.
Gold	lh_gold + nb_03_gold_star_schema	Kimball star schema: 9 conformed dimensions, 11 facts, promotion attribution, aggregated labour, DQ summary, RLS mapping table. V-Order enabled for Direct Lake reads.
AI	nb_05_ml_demand_forecast, MLflow experiment dairyco-demand-forecast, registered model dairyco-demand-forecast-hgb	SKU × month demand model with rolling-origin backtest vs MRP forecast; writes forecasts and a stock-rebalancing recommendation to gold.
Orchestration	Data pipeline p1_dairyco_daily	Bronze → Silver (DQ gate) → Gold → ML, batch_id = pipeline RunId threaded through every layer, 2 retries / 2-hour timeout per activity.
Semantic model	DairyCo Analytics (Direct Lake on lh_gold)	23 tables, 43 relationships, 95 documented measures in display folders, time intelligence, dynamic RLS + OLS. Authored as TMDL (code) and deployed through the Fabric REST API.
Report	DairyCo Management (3 pages)	Executive Performance, Commercial, Operations — bound live to the semantic model.
ALM	DairyCo_v2_Dev → DairyCo_v2_Prod, GitHub repo	Dev/Prod workspaces, source in Git, deployment pipeline between stages.

1.2 Key decisions, alternatives and why

Decision	Chosen	Credible alternative	Why this choice here
Storage & compute	Lakehouse + PySpark notebooks (3 lakehouses)	Fabric Warehouse (T-SQL) for gold; Dataflow Gen2 for transforms	Sources include semi-structured JSONL and need MERGE, regex SKU mapping and DQ logging — Spark handles all in one engine. Warehouse is the right call if the BI team is T-SQL-first; gold could move there without changing the semantic layer. Dataflow Gen2 is low-code but harder to test, version and parameterise at 22 entities.
Layer separation	One lakehouse per layer	One schema-enabled lakehouse with bronze/silver/gold schemas	A lakehouse is a security boundary: raw payroll never leaves bronze/silver, which only data engineers can read; business users get lh_gold via the semantic model only. Schemas are tidier but share one permission surface.

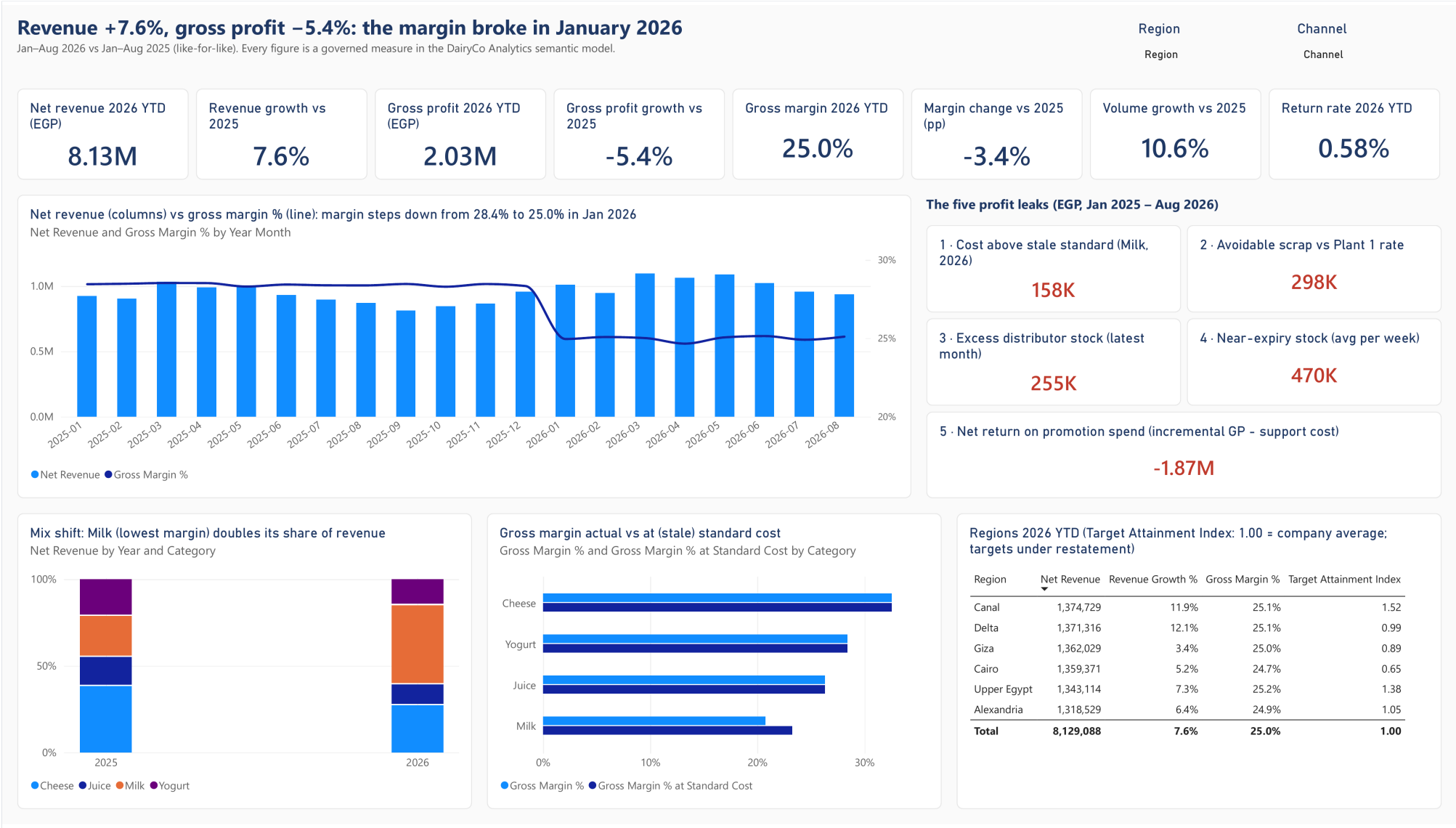
Decision	Chosen	Credible alternative	Why this choice here
Ingestion pattern	Config-driven notebook + landing folders	One Copy activity per source; Mirroring / shortcuts to source DBs	The files are extracts, so a landing zone mirrors how SFTP/API drops arrive. Adding a source = one JSON line, no code. In production, real ERP/CRM DBs would use Fabric Mirroring or Copy jobs into the same landing contract — nothing downstream changes.
Incremental processing	Watermark on _ingested_at + Delta MERGE on business key (Sales); full-refresh for master/snapshot extracts	CDC via Mirroring; Materialized Lake Views with incremental refresh	Sales is the only high-volume, append-mostly entity; MERGE also absorbs late corrections. Demonstrated: initial load to 31 Jul 2026, then an August drop (3,723 lines) merged → 75,354 rows, watermark advanced, re-run is a no-op. Masters are small; full refresh is simpler and self-healing.
Data quality	Flag, don't fix — dq_issues + history, hard gate only on value-breaking rules	Great Expectations / Purview DQ	The brief asks to document anomalies rather than silently correct business ambiguities. Duplicates are <i>mapped</i> (auditable) not deleted; relabelled regions keep the raw label. Blocking only SAL-03 (net ≠ gross – discount) and PRD-01 (actual ≠ good + scrap) avoids a pipeline that never runs.
Model storage mode	Direct Lake	Import; DirectQuery	No scheduled refresh or data duplication; gold Delta tables (V-Ordered) are read directly and new data is picked up automatically. Import would be chosen for complex calculated columns or non-Fabric sources; DirectQuery only for true real-time.
Model authoring	TMDL generated from code, deployed via REST	Power BI Desktop by hand	Every table/column/measure carries a description (feeds Copilot/data agents), names are consistent, and the model is diff-able in Git and re-deployable to any stage by one command.
Security	Dynamic RLS on Region + OLS on labour cost + labour aggregated before gold	Static role per region; RLS in the SQL endpoint	One role serves all regional managers via a mapping table (production: synced from Entra ID groups). Payroll protection is layered: employee pay never reaches gold (min. cell size 5), cost columns are object-level secured for regional users, silver/bronze are restricted workspaces/lakehouses.
AI	Gradient-boosted trees, MLflow tracked, batch scoring into gold	Prophet/ETS per SKU; Azure ML AutoML	24 SKUs × 20 months is too short for per-series statistical models to beat a pooled model; a pooled tree model learns cross-SKU patterns. MLflow in Fabric gives lineage and model registry at no extra cost.
Capacity	F2 (dev/demo)	F64 for production	11 MB of data runs comfortably on F2 starter pools. Production with daily loads, several hundred report users and Copilot would be sized at F64 (also removes the need for Pro licences for viewers).

1.3 Grain, keys and conformed dimensions

- All keys are integers; dates join on date_key (yyyymmdd). Month-grain facts (targets, forecast, production, labour, distributor) use the first day of the month, so every fact shares the one date dimension.
- **Region** is conformed across Sales, Returns, Targets, Promotions, Inventory, Distributor, CRM and ML rebalancing and is the RLS anchor. Region *names* are denormalised onto Customer/Distributor/Warehouse instead of relating those dimensions to **Region**, which would create ambiguous filter paths.
- **Production Unit** (plant × line × shift) is conformed between production orders and aggregated labour, which is what allows scrap, downtime, overtime and absence to be compared cell by cell.
- Targets are at month × region × category; the category link to **Product** is virtual (**TREATAS**) so no bridge table is needed.

Management report (live in Fabric)

Page 1



Commercial: growth sits in five Milk SKUs, Nile is stuffing its channel, promotions don't pay back

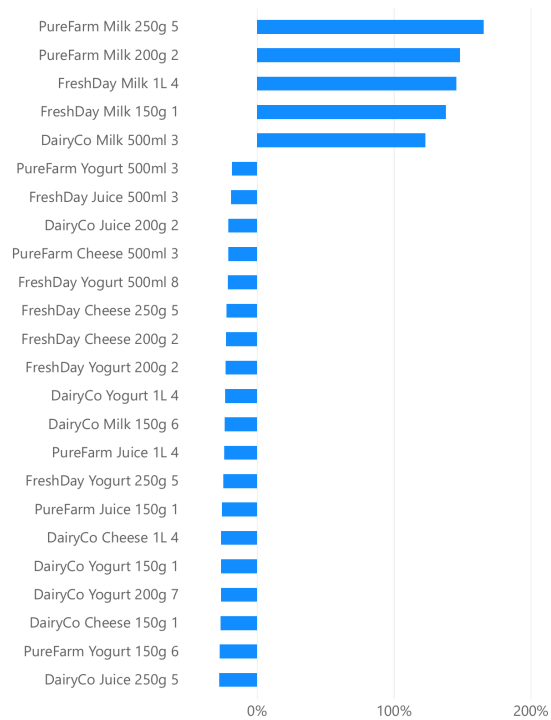
Sell-in (ERP) vs sell-out (distributor platform) · promotion attribution by product + region + date · CRM coverage: Modern Trade & Key Accounts only

Region
Region
Channel
Channel

Net revenue (EGP) (2026)	Revenue growth YoY (2026)	Avg selling price (2026)	Invoice discount (2026)	Revenue on promotion (2026)	Distributor sell-through (2026)	Open CRM pipeline	Pipeline past close date
8.13M	7.6%	33.27	3.6%	0.7%	82.6%	91.95M	90.7%

Revenue growth by SKU, 2026 vs 2025: Milk 1001–1005 +120–170%, the rest -18% to -27%

Revenue Growth % by Product

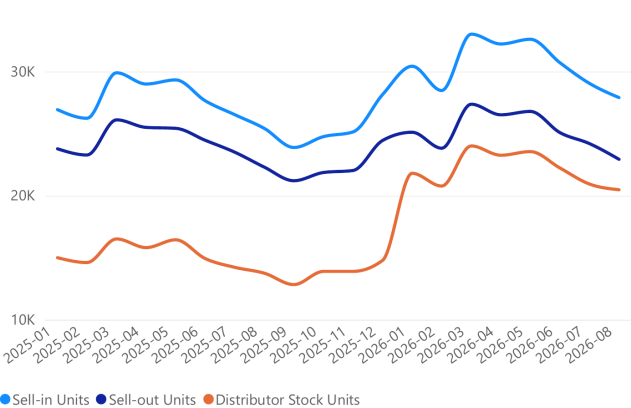


Distributor scorecard 2026 YTD — judge distributors on sell-out and stock cover, not sell-in

Distributor	Net Revenue	Revenue Growth %	Sell-in Units	Sell-out Units	Sell-through %	Distributor Months of Cover	Excess Channel Stock Value	Return Rate %
Nile Distribution	1,359,371	5.2%	41,117	23,750	57.8%	2.75	198,492	0.60%
Delta Trade	1,371,316	12.1%	40,921	35,954	87.9%	0.66	14,253	0.56%
Mediterranean Foods	1,318,529	6.4%	39,697	34,797	87.7%	0.66	12,654	0.51%
Canal Consumer	1,374,729	11.9%	41,280	36,127	87.5%	0.65	13,309	0.62%
Giza Markets	1,362,029	3.4%	41,021	35,915	87.6%	0.63	9,122	0.66%
Upper Egypt Distribution	1,343,114	7.3%	40,324	35,229	87.4%	0.61	7,375	0.52%
Total	8,129,088	7.6%	244,360	201,772	82.6%	0.89	255,205	0.58%

Sell-in vs sell-out vs distributor stock (select a distributor above)

Sell-in Units, Sell-out Units and Distributor Stock Units by Year Month



Promotion ROI — 0 of 37 measurable promotions cover their support cost

Promotion	Promotion Support Cost	Average Volume Lift %	Net Promotion Return
PRM-9999	120,000	0.0%	-122,213
PRM-5022	44,539	34.8%	-44,396
PRM-5013	42,911	-7.7%	-43,001
PRM-5021	42,563	-45.0%	-42,741
PRM-5045	42,377		-42,422
PRM-5049	42,119		-41,859
PRM-5028	41,700	-32.7%	-41,841
PRM-5059	41,431		-41,273
PRM-5008	40,882	-6.9%	-40,889
PRM-5019	40,601	-37.1%	-40,788
PRM-5041	38,880	-29.6%	-39,014
PRM-5010	38,417	-18.8%	-38,576
PRM-5048	38,488	16.5%	-38,351
Total	1,869,103	-11.6%	-1,873,631

Operations: plan attainment hides Plant 2 / Shift C scrap and a copy-paste forecast

Production plan attainment on actual vs good output · forecast bias by category · weekly warehouse snapshots · labour aggregated to plant/line/shift (no individual pay)

Year
Year
Plant
Plant

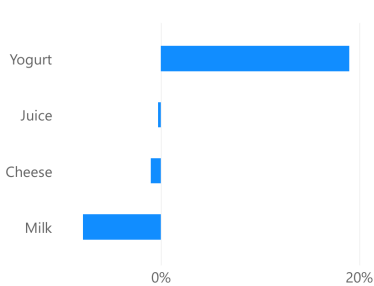
Plan attainment (actual)	Plan attainment (good)	Scrap rate	OT hrs / employee / month	MRP forecast error (WAPE)	MRP forecast bias	Stock near expiry	Stockout proxy*
99.8%	95.3%	4.49%	22.2	10.3%	2.5%	27.8%	0.06%

Plant × shift: scrap, downtime, overtime and absence move together (avoidable scrap vs Plant 1 average)

Plant	Shift	Production Plan Attainment %	Scrap Rate %	Overtime Hours per Employee	Absence Rate %	Avoidable Scrap Cost
Plant 2	Shift C	102.6%	8.99%	36.0	8.1%	102,598
Plant 2	Shift B	102.9%	5.52%	27.9	5.7%	75,180
Plant 2	Shift A	102.2%	5.43%	28.2	5.6%	66,156
Plant 1	Shift C	97.7%	5.41%	18.0	5.3%	48,455
Plant 1	Shift A	96.9%	1.85%	10.2	3.0%	2,934
Plant 1	Shift B	97.4%	1.76%	10.3	2.9%	2,908
Total		99.8%	4.49%	22.2	5.2%	298,231

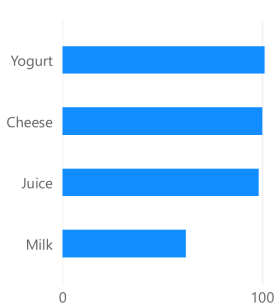
MRP forecast bias by category (+ = over-forecast)

Forecast Bias % by Category



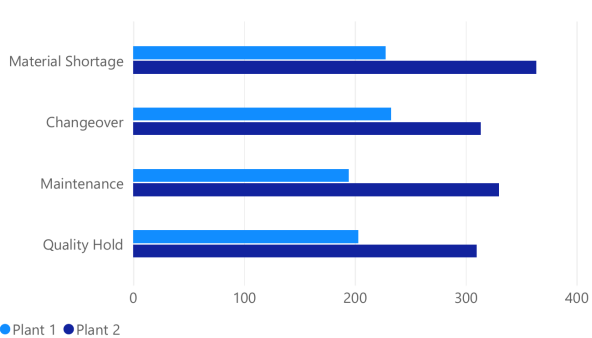
Warehouse days of cover by category

Days of Cover by Category



Downtime hours by reason and plant

Downtime Hours by Downtime Reason and Plant



AI prototype — forecast error, 6-month out-of-sample backtest

MRP forecast (today)	Bias-corrected rule
9.9%	5.9%
ML correction model	Stock above target cover
5.8%	1.50M

Target cover = 50% of shelf life. Stock and invoice volumes appear to be on different scales; use for allocation (which DC/SKU), not absolute value.

Data trust: latest data-quality checks (flagged, not silently fixed). *Stockout = proxy (no order/backorder data)

Rule Id	Rule	Severity	DQ Violations
SAL-04	COGS deviates from Qty x StandardCost (stale standard cost)	High	14,544
INV-01	Available <> OnHand - Reserved	Medium	12,048
DST-03	Distributor-reported sell-out <> sum of outlet sell-out	Medium	2,820
DST-02	Inventory roll-forward does not balance (opening+receipts-sellout<>closing)	Medium	2,718
INV-03	Near-expiry stock (<=3 days) still status 'Available'	Medium	1,345
DST-01	Distributor SKU not in agreed D-#### format	Medium	1,308
SAL-02	Invoice booked to a duplicate customer ID	High	819
CRM-03	Open opportunity past expected close date	Low	359
SAL-01	Non-standard region label on invoice line	Medium	185
Total			36,565

2. Exploratory data analysis and data quality

The EDA was run on the raw extracts before any modelling, using DuckDB locally. It was then re-run through the Fabric pipeline, so every data-quality finding below is also a live rule in `nb_02_silver_transform`, with its violation count on the Operations page.

2.1 Profile of the sources

Source	Files	Grain	Range	Notes
ERP + WMS	9	invoice line (75,354), return line (1,659), weekly batch snapshot (12,503), month × region × category targets	Jan 2025 – Aug 2026	Net = Gross – Discount on 100% of lines; COGS = Qty × Standard Cost everywhere except Milk SKUs 1001–1005 in 2026 (× 1.06).
Salesforce CRM	3	account (180), opportunity (780), activity (3,234)	Jan 2025 – Aug 2026	Covers Modern Trade and Key Accounts only (23.7% of ERP revenue).
Distributor platform	3	month × distributor × SKU × outlet (11,605 JSONL), month × distributor × SKU (2,880 JSONL), 540 outlets	Jan 2025 – Aug 2026	Receipts = ERP sell-in exactly; outlets cannot be linked to ERP customers.
Production + MRP	4	order = month × plant × line × shift × SKU (960)	Jan 2025 – Aug 2026	Actual = Good + Scrap on 100%; downtime log reconciles 100%.
HR + Payroll	3	employee (342), employee × month (6,840 each)	Jan 2025 – Aug 2026	Total labour cost = base + OT + allowances on 100%.

All sources cover the same 20 months, so **year-on-year is only like-for-like for Jan–Aug** (2026 vs 2025).

2.2 Data-quality issues (all flagged in `dq_issues`, none silently fixed)

Rule	Issue	Rows	Treatment	Business impact
CUS-01 / SAL-02	8 duplicate customers (upper-case copies, IDs 20721–20728)	8 customers, 819 invoice lines (≈201k EGP)	Mapped to the proper-case master; both IDs kept	Customer count 728 → 720; per-customer KPIs were split.
SAL-01	Non-standard region labels on invoices ("CAIRO", "Alex.", "UpperEgypt")	185 lines (46.8k EGP)	Region taken from the customer master; raw label retained	Regional revenue and target attainment were understated.
SAL-04	COGS ≠ Qty × Standard Cost	14,544 lines (Milk 1001–1005, 2026)	Kept actual COGS; Standard COGS stored alongside	Leak 1: standard costs are stale.
CRM-01 / CRM-02	CRM CustomerID stored as float text ("20290.0"); 10 blank	170 / 10 accounts	Cast to integer; blanks matched by exact name (10/10)	CRM ↔ ERP join now 100% (was 94%).
CRM-03	Open opportunities past expected close date	345	Flagged is_stale	91.9M "open pipeline" is mostly stale.
DST-01	Distributor SKU "UNKNOWN-1013/1018/1023"	1,308 sell-out rows	ERP product recovered from the numeric suffix; flagged	Without the fix, 3 SKUs vanish from sell-out.
DST-02	Distributor stock roll-forward does not balance	2,718 of 2,736	Flagged; reported closing stock used	Channel stock figures are indicative only.

Rule	Issue	Rows	Treatment	Business impact
DST-03	Distributor-reported sell-out $\neq$ sum of outlet sell-out	2,820 of 2,880 cells	Reported figure used for cover, outlet figure for value	Two versions of the same KPI; needs a data contract.
INV-01	Available $\neq$ On-hand – Reserved (off by 1)	12,048 of 12,503	Recomputed Qty Available; reported value kept	Minor, but a sign of a WMS calculation bug.
INV-02	Negative on-hand	8	Floored at 0 for valuation; flagged	Integrity issue.
INV-03	Stock $\leq$ 3 days from expiry still status "Available"	1,345	Status ignored; near-expiry derived from dates	The status field cannot be used for stock-risk decisions.
RET-01 / RET-02	Returns dated after the last sale / before the invoice was loaded	22 / late-arriving	Warning, not blocking	Return rate depends on the dating convention (return date used).
HR-01	Payroll paid before the employee's hire date	112 employee-months (28 people)	Flagged	Payroll control issue for HR/Finance.
CUS-02	Territory prefix "T-CA" used for Cairo and Canal	1 prefix	Flagged	Territory codes are not unique.
—	Production order 600001 missing	1	Noted	Sequence gap.
—	Targets $\approx$ 24 $\times$ actual revenue; fixed implied price 35 EGP	all 480 rows	Relative index only	Absolute target attainment (~4%) is meaningless.

2.3 Patterns, trends and anomalies

- **Trend and seasonality.** Revenue peaks March–May and bottoms in September. There is no calendar gap: all 608 days have sales, with about 12% fewer lines on Fridays.
- **Step change in January 2026.** Monthly gross margin is 28.3–28.5% in every month of 2025 and 24.7–25.1% in every month of 2026. This is a structural break, not noise.
- **Mix change.** Milk SKUs 1001–1005 grew 123–166% YoY while every other SKU (including Milk 1006) fell 18–27%. List prices never changed; average selling price fell about 3% from mix alone.
- **Distributor 301.** From January 2026 its sell-out falls about 30% while its receipts rise, and its closing stock roughly triples. The other five distributors stay at about 0.6 months of cover.
- **Promotion 9999.** It is the only "Deep Discount" (25%, SKU 1022, Cairo, Apr–May 2026, 120k support cost against 15–45k for others) and the only promotion whose discount appears on invoices. It had zero volume lift.
- **Plant 2 / Shift C.** Scrap is 6.3% at Plant 2 vs 2.7% at Plant 1, and Shift C reaches 7.1% (8.8–9.3% at Plant 2). Overtime and absence follow the same cells (correlation 0.89 across cells) but are flat over time.
- **Forecast.** The production plan equals the forecast minus about one unit. The forecast is biased by category (Milk –8%, Yogurt +19%).
- **Cross-source scale.** Targets (24 $\times$), payroll (about 4.5M/month vs about 1M revenue), CRM won value (12 $\times$) and warehouse stock (80–130 days of cover for 14-day milk) are not on the same scale as invoice sales. **Assumption:** invoice data is the reliable commercial record, other sources are used for relative comparison, and every affected measure says so in its description.

2.4 Material assumptions

1. **Revenue** is ERP net sales (after on-invoice discount, before returns). Returns are reported separately and dated by return date.

2. **Region** is the customer-master region, not the free-text invoice label.
3. **Near-expiry** means 25% or less of shelf life remaining.
4. **Promotion baseline** is the same product and region in the 8 weeks before the promotion, excluding days in other promotions. A promotion is "measurable" when its baseline has at least 28 trading days.
5. **Benchmark scrap** is the Plant 1 shifts A/B rate in the same period (internal best practice).
6. **Normal distributor cover** is 0.6 months (the peer level observed at distributors 302–306).

6. Semantic model reference: DairyCo Analytics

Direct Lake on lh_gold. It is generated from scripts/build_model.py as TMDL (semantic-model/) and deployed through the Fabric REST API. It has 23 tables, 43 relationships (all single-direction, many-to-one, integer keys), 95 measures and 2 roles.

6.1 Tables and grain

Table	Source (gold)	Description / grain
Date	dim_date	Calendar 2025-2026 (daily). Egyptian weekend = Friday/Saturday. 'Is Actuals Period' marks dates up to the last loaded invoice date and is used to keep year-on-year comparisons like-for-like.
Product	dim_product	ERP product master (24 SKUs). Standard Cost is the ERP master value; it is stale for Milk SKUs 1001-1005 from Jan 2026 (see Cost Variance vs Standard).
Region	dim_region	Conformed sales region (6 regions + Unknown). Row-level security is applied on this table; every regional fact relates to it directly.
Customer	dim_customer	Master customers after de-duplication (8 upper-case duplicate IDs merged into their proper-case master). Region is denormalised (no relationship to Region, to avoid ambiguous paths).
Distributor	dim_distributor	Third-party distributors (one per region).
Warehouse	dim_warehouse	DairyCo distribution centres (one per region).
Promotion	dim_promotion	Trade promotions from ERP plus a 'No promotion' member (key -1) for unpromoted invoice lines.
Production Unit	dim_production_unit	Plant x line x shift (18 units) - conformed between production orders and aggregated labour.
Sales	fact_sales	ERP invoice lines (grain: invoice line). Region comes from the customer master, customer is the de-duplicated master, promotion is attributed by product + region + invoice date.
Returns	fact_returns	Customer return lines (grain: return line). Dated by return date; returns can arrive after the sales period.
Sales Target	fact_sales_target	Monthly revenue and volume targets by region x category. WARNING: targets are ~24x actual revenue (implied price fixed at 35 EGP); use only relative/indexed attainment until Finance restates them.
Promotion Performance	fact_promo_performance	One row per promotion, dated by promotion start. Baseline = same product+region daily run-rate in the 8 weeks before the promotion (excluding other promotions). Incremental GP = promo-window GP minus baseline GP.
Inventory Snapshot	fact_inventory_snapshot	Weekly (Sunday) warehouse stock by batch. Semi-additive: stock measures take the latest snapshot in context. Near-expiry = 25% or less of shelf life remaining (business assumption).
Distributor Month	fact_distributor_month	Month x distributor x SKU: ERP sell-in, distributor-reported receipts, sell-out and closing stock. Distributor SKUs mapped to ERP products (UNKNOWN-#### recovered).
Demand Forecast	fact_demand_forecast	MRP monthly demand forecast by SKU (the incumbent forecast; the production plan copies it).
Production	fact_production	Production orders (grain: order = month x plant x line x shift x SKU).
Labour	fact_labour_month	Payroll and attendance AGGREGATED to month x plant x line x shift (minimum 5 employees per cell). No employee-level pay is in the model. Cost columns are hidden from the Regional Manager role (OLS).

Table	Source (gold)	Description / grain
CRM Opportunity	fact_crm_opportunity	Salesforce opportunities (grain: opportunity), dated by created month. CRM covers Modern Trade and Key Accounts only.
CRM Activity	fact_crm_activity	Salesforce activities (grain: activity).
ML Demand Forecast	ml_demand_forecast	Output of nb_05 (MLflow-tracked). record_type 'backtest' = rolling-origin out-of-sample months; 'forecast' = next month.
Stock Rebalancing	ml_stock_rebalancing	Warehouse x SKU recommendation from nb_05: target stock = ML forecast daily demand x (50% of shelf life); gap = on hand - target.
Data Quality	dq_summary	Latest result of every silver data-quality rule (flag, don't silently fix).
Security User Region	security_user_region	RLS mapping: user principal name -> permitted region. Maintained by the BI owner (production: synced from Entra ID groups).

6.2 Relationships

All relationships are many-to-one and single-direction, from fact to dimension, on integer keys.

Fact	Related dimensions
Sales	Date (via Date Key), Customer (via Customer Key), Product (via Product Key), Distributor (via Distributor Key), Region (via Region Key), Promotion (via Promotion Key)
Returns	Date (via Date Key), Customer (via Customer Key), Product (via Product Key), Region (via Region Key), Distributor (via Distributor Key)
Sales Target	Date (via Date Key), Region (via Region Key)
Promotion Performance	Promotion (via Promotion Key), Product (via Product Key), Region (via Region Key), Date (via Date Key)
Inventory Snapshot	Date (via Date Key), Warehouse (via Warehouse Key), Product (via Product Key), Region (via Region Key)
Distributor Month	Date (via Date Key), Distributor (via Distributor Key), Product (via Product Key), Region (via Region Key)
Demand Forecast	Date (via Date Key), Product (via Product Key)
Production	Date (via Date Key), Production Unit (via Production Unit Key), Product (via Product Key)
Labour	Date (via Date Key), Production Unit (via Production Unit Key)
CRM Opportunity	Date (via Date Key), Customer (via Customer Key), Region (via Region Key)
CRM Activity	Date (via Date Key), Customer (via Customer Key), Region (via Region Key)
ML Demand Forecast	Date (via Date Key), Product (via Product Key)
Stock Rebalancing	Warehouse (via Warehouse Key), Product (via Product Key), Region (via Region Key)

6.3 Security roles

Executive. Executives and central functions: enterprise-wide view, no row filters.

Regional Manager. Regional managers: rows limited to regions mapped to their UPN in Security User Region. Manufacturing and aggregated workforce KPIs are company-wide and visible, but labour cost columns are hidden (OLS). - RLS on *Region*: 'Region'[Region Key] IN CALCULATETABLE(VALUES('Security User Region'[Region Key]), 'Security User Region'[User UPN] = USERPRINCIPALNAME()) - RLS on *Customer*: 'Customer'[Customer Region Key] IN CALCULATETABLE(VALUES('Security User Region'[Region Key]), 'Security User Region'[User UPN] = USERPRINCIPALNAME()) - RLS on *Distributor*: 'Distributor'[Distributor Region Key] IN CALCULATETABLE(VALUES('Security User Region'[Region Key]), 'Security User Region'[User UPN] = USERPRINCIPALNAME()) - RLS on *Warehouse*: 'Warehouse'[Warehouse Region Key] IN CALCULATETABLE(VALUES('Security User Region'[Region Key]), 'Security User Region'[User UPN] = USERPRINCIPALNAME()) - RLS on *Security User Region*: 'Security User Region'[User UPN] = USERPRINCIPALNAME() - OLS on *Labour*: Overtime Cost Amount, Base Salary Amount, Allowances Amount, Total Labour Cost Amount = none

6.4 Measure catalogue

Sales

Measure	Definition	DAX
Net Revenue	Invoiced net sales (gross less on-invoice discount), before returns. Official revenue KPI; EGP.	SUM('Sales'[Net Sales])
Gross Sales Amount	Quantity x list price before discount; EGP.	SUM('Sales'[Gross Sales])
Invoice Discount	On-invoice discount; EGP.	SUM('Sales'[Discount Amount])
Invoice Discount %	On-invoice discount as a share of gross sales.	DIVIDE([Invoice Discount], [Gross Sales Amount])
Sales Volume	Units invoiced (packs, not litres/kg).	SUM('Sales'[Quantity])
Average Selling Price	Net revenue per unit; moves with mix because list prices never changed.	DIVIDE([Net Revenue], [Sales Volume])
Active Customers	Distinct master customers invoiced.	DISTINCTCOUNT('Sales'[Customer Key])
Cost of Goods Sold	Actual COGS booked on invoice lines; EGP.	SUM('Sales'[COGS])
Gross Profit	Net revenue less actual COGS; EGP.	[Net Revenue] - [Cost of Goods Sold]
Gross Margin %	Gross profit / net revenue.	DIVIDE([Gross Profit], [Net Revenue])
Standard Cost of Goods Sold	Quantity x ERP standard cost - what margin reporting based on master data assumes.	SUM('Sales'[Standard COGS])
Cost Variance vs Standard	Actual minus standard COGS. Positive = costs running above the ERP standard (Leak 1: stale Milk standard costs from Jan 2026).	[Cost of Goods Sold] - [Standard Cost of Goods Sold]

Measure	Definition	DAX
Gross Margin % at Standard Cost	The margin the business believes it makes if it prices off stale standard costs.	<code>DIVIDE([Net Revenue] - [Standard Cost of Goods Sold], [Net Revenue])</code>
Net Revenue PY	Net revenue for the same dates one year earlier, limited to dates that have actuals this year (like-for-like).	<code>VAR _dates = CALCULATETABLE(VALUES('Date'[Date]), 'Date'[Is Actuals Period] = TRUE()) RETURN CALCULATE([Net Revenue], DATEADD(_dates, -1, YEAR))</code>
Revenue Growth %	Like-for-like year-on-year growth for the selected year (defaults to the latest year with actuals). Only 2026 Jan-Aug vs 2025 Jan-Aug is comparable (data starts Jan 2025).	<code>VAR _yr = IF(HASONEVALUE('Date'[Year]), VALUES('Date'[Year]), CALCULATE(MAX('Date'[Year]), 'Date'[Is Actuals Period] = TRUE(), REMOVEFILTERS('Date')))) VAR _cy = CALCULATE([Net Revenue], 'Date'[Is Actuals Period] = TRUE(), 'Date'[Year] = _yr) VAR _py = CALCULATE([Net Revenue PY], 'Date'[Year] = _yr) RETURN IF(NOT ISBLANK(_py), DIVIDE(_cy - _py, _py))</code>
Gross Profit PY	Gross profit, same dates last year (like-for-like).	<code>VAR _dates = CALCULATETABLE(VALUES('Date'[Date]), 'Date'[Is Actuals Period] = TRUE()) RETURN CALCULATE([Gross Profit], DATEADD(_dates, -1, YEAR))</code>
Gross Profit Growth %	Like-for-like year-on-year gross profit growth.	<code>VAR _yr = IF(HASONEVALUE('Date'[Year]), VALUES('Date'[Year]), CALCULATE(MAX('Date'[Year]), 'Date'[Is Actuals Period] = TRUE(), REMOVEFILTERS('Date')))) VAR _cy = CALCULATE([Gross Profit], 'Date'[Is Actuals Period] = TRUE(), 'Date'[Year] = _yr) VAR _py = CALCULATE([Gross Profit PY], 'Date'[Year] = _yr) RETURN IF(NOT ISBLANK(_py), DIVIDE(_cy - _py, _py))</code>
Gross Margin % PY	Gross margin, same dates last year.	<code>DIVIDE([Gross Profit PY], [Net Revenue PY])</code>
Gross Margin Change pp	Change in gross margin vs the same period last year (percentage points); defaults to the latest year with actuals.	<code>VAR _yr = IF(HASONEVALUE('Date'[Year]), VALUES('Date'[Year]), CALCULATE(MAX('Date'[Year]), 'Date'[Is Actuals Period] = TRUE(), REMOVEFILTERS('Date')))) VAR _cy = CALCULATE([Gross Margin %], 'Date'[Is Actuals Period] = TRUE(), 'Date'[Year] = _yr) VAR _py = CALCULATE([Gross Margin % PY], 'Date'[Year] = _yr) RETURN IF(NOT ISBLANK(_py), _cy - _py)</code>
Sales Volume PY	Units, same dates last year (like-for-like).	<code>VAR _dates = CALCULATETABLE(VALUES('Date'[Date]), 'Date'[Is Actuals Period] = TRUE()) RETURN CALCULATE([Sales Volume], DATEADD(_dates, -1, YEAR))</code>
Sales Volume Growth %	Like-for-like year-on-year unit growth.	<code>VAR _yr = IF(HASONEVALUE('Date'[Year]), VALUES('Date'[Year]), CALCULATE(MAX('Date'[Year]), 'Date'[Is Actuals Period] = TRUE(), REMOVEFILTERS('Date')))) VAR _cy = CALCULATE([Sales Volume], 'Date'[Is Actuals Period] = TRUE(), 'Date'[Year] = _yr) VAR _py = CALCULATE([Sales Volume PY], 'Date'[Year] = _yr) RETURN IF(NOT ISBLANK(_py), DIVIDE(_cy - _py, _py))</code>
Net Revenue YTD	Calendar year-to-date net revenue.	<code>TOTALYTD([Net Revenue], 'Date'[Date])</code>
Promoted Revenue %	Share of net revenue sold inside a promotion window.	<code>DIVIDE(CALCULATE([Net Revenue], 'Sales'[Promotion Key] <> -1), [Net Revenue])</code>
Net Revenue after Returns	Net revenue less the value of goods returned in the period (return-date basis).	<code>[Net Revenue] - [Return Value]</code>

Returns

Measure	Definition	DAX
Return Value	Value of returned goods at invoice net price; EGP.	SUM('Returns'[Return Value Amount])
Returned Units	Units returned.	SUM('Returns'[Returned Quantity])
Return Rate %	Return value / net revenue, both by their own date (return date vs invoice date).	DIVIDE([Return Value], [Net Revenue])

Sales Target

Measure	Definition	DAX
Revenue Target	Revenue target; category filter flows from Product[Category] via TREATAS. Source targets are on a different scale from actuals.	CALCULATE(SUM('Sales Target'[Revenue Target Amount]), TREATAS(VALUES('Product'[Category]), 'Sales Target'[Target Category]))
Target Attainment %	UNRELIABLE AS ABSOLUTE: ~4% because targets are ~24x actuals. Shown for completeness; use Target Attainment Index.	DIVIDE([Net Revenue], [Revenue Target])
Target Attainment Index	Attainment relative to the company average in the same period (1.00 = average). Scale-free, so usable while target scale is unresolved.	VAR _att = [Target Attainment %] VAR _company = CALCULATE([Target Attainment %], REMOVEFILTERS('Region'), REMOVEFILTERS('Product'), REMOVEFILTERS('Customer'), REMOVEFILTERS('Distributor')) RETURN DIVIDE(_att, _company)

Promotion Performance

Measure	Definition	DAX
Promotion Support Cost	Trade support paid for the promotions in context; EGP.	SUM('Promotion Performance'[Support Cost])
Incremental Gross Profit	Gross profit above the pre-promotion baseline; EGP.	SUM('Promotion Performance'[Incremental GP])
Net Promotion Return	Incremental gross profit less support cost; negative = value destroyed.	SUM('Promotion Performance'[Net Promo Return])
Promotion ROI %	Net promotion return / support cost.	DIVIDE([Net Promotion Return], [Promotion Support Cost])
Promotions Run	Number of promotions.	COUNTROWS('Promotion Performance')
Measurable Promotions	Promotions with at least 28 baseline trading days in the 8 weeks before start.	CALCULATE(COUNTROWS('Promotion Performance'), 'Promotion Performance'[Baseline Measurable] = TRUE()) + 0
Promotions Paying Back	Promotions with a measurable baseline whose incremental GP exceeded support cost.	CALCULATE(COUNTROWS('Promotion Performance'), 'Promotion Performance'[Net Promo Return] > 0, 'Promotion Performance'[Baseline Measurable] = TRUE()) + 0
Average Volume Lift %	Mean unit uplift vs baseline across measurable promotions.	CALCULATE(AVERAGE('Promotion Performance'[Volume Lift]), 'Promotion Performance'[Baseline Measurable] = TRUE())

Measure	Definition	DAX
Discount Applied on Invoice %	Average discount actually on invoices during the promotion window (compare with stated discount).	AVERAGE('Promotion Performance'[Discount Applied On Invoice])
Promotion Incremental Units	Units above baseline.	SUM('Promotion Performance'[Incremental Units])

Inventory Snapshot

Measure	Definition	DAX
Inventory Units	Units on hand at the latest snapshot in context (semi-additive).	VAR _k = CALCULATE(MAX('Inventory Snapshot'[Date Key])) RETURN CALCULATE(SUM('Inventory Snapshot'[Qty On Hand]), 'Inventory Snapshot'[Date Key] = _k)
Inventory Value	Stock on hand at standard cost, latest snapshot in context; EGP.	VAR _k = CALCULATE(MAX('Inventory Snapshot'[Date Key])) RETURN CALCULATE(SUM('Inventory Snapshot'[Stock Value]), 'Inventory Snapshot'[Date Key] = _k)
Near-Expiry Stock Value	Stock with 25% or less of shelf life left at the latest snapshot, at standard cost.	VAR _k = CALCULATE(MAX('Inventory Snapshot'[Date Key])) RETURN CALCULATE(SUM('Inventory Snapshot'[Stock Value]), 'Inventory Snapshot'[Date Key] = _k, 'Inventory Snapshot'[Is Near Expiry] = TRUE())
Near-Expiry Stock %	Share of stock value close to expiry.	DIVIDE([Near-Expiry Stock Value], [Inventory Value])
Average Near-Expiry Stock Value	Average near-expiry stock value per weekly snapshot across the period - the recurring exposure.	AVERAGEX(VALUES('Inventory Snapshot'[Date Key]), CALCULATE(SUM('Inventory Snapshot'[Stock Value]), 'Inventory Snapshot'[Is Near Expiry] = TRUE()))
Days of Cover	Latest stock / average daily units sold in the period. CAVEAT: warehouse stock and invoice volume look to be on different scales.	VAR _units = [Inventory Units] VAR _days = CALCULATE(COUNTROWS('Date'), 'Date'[Is Actuals Period] = TRUE()) VAR _daily = DIVIDE(CALCULATE([Sales Volume], 'Date'[Is Actuals Period] = TRUE()), _days) RETURN DIVIDE(_units, _daily)
Stockout Proxy %	PROXY, not a true stockout rate: share of warehouse x SKU x week snapshots with zero available stock. A real measure needs order lines (requested vs fulfilled) and daily/outlet stock.	DIVIDE(CALCULATE(COUNTROWS('Inventory Snapshot'), 'Inventory Snapshot'[Is Zero Available] = TRUE()), COUNTROWS('Inventory Snapshot'))

Distributor Month

Measure	Definition	DAX
Sell-in Units	Units received by distributors (equals ERP sell-in).	SUM('Distributor Month'[Receipts Qty])
Sell-out Units	Units distributors report selling to outlets.	SUM('Distributor Month'[Sellout Qty Reported])
Sell-out Value	Outlet-level sell-out value at distributor prices.	SUM('Distributor Month'[Sellout Value])
Sell-through %	Sell-out / sell-in. Below ~85% (the peer norm) means stock is building in the channel.	DIVIDE([Sell-out Units], [Sell-in Units])

Measure	Definition	DAX
Distributor Stock Units	Distributor closing stock at the latest month in context (semi-additive).	<code>VAR _k = CALCULATE(MAX('Distributor Month'[Date Key])) RETURN CALCULATE(SUM('Distributor Month'[Closing Inventory Qty]), 'Distributor Month'[Date Key] = _k)</code>
Distributor Months of Cover	Closing stock / that month's sell-out. Peers run at ~0.6 months.	<code>VAR _k = CALCULATE(MAX('Distributor Month'[Date Key])) VAR _stock = [Distributor Stock Units] VAR _sellout = CALCULATE([Sell-out Units], 'Distributor Month'[Date Key] = _k) RETURN DIVIDE(_stock, _sellout)</code>
Distributor Stock Value	Closing distributor stock valued at DairyCo's sell-in price, latest month in context.	<code>VAR _k = CALCULATE(MAX('Distributor Month'[Date Key])) RETURN SUMX(FILTER('Distributor Month', 'Distributor Month'[Date Key] = _k), 'Distributor Month'[Closing Inventory Qty] * 'Distributor Month'[Sell In Unit Price])</code>
Excess Channel Stock Value	Distributor stock above 0.6 months of cover (peer norm), at sell-in price - revenue pulled forward that must still sell through.	<code>VAR _k = CALCULATE(MAX('Distributor Month'[Date Key])) VAR _benchmarkMonths = 0.6 RETURN SUMX(FILTER('Distributor Month', 'Distributor Month'[Date Key] = _k), MAX(0, 'Distributor Month'[Closing Inventory Qty] - _benchmarkMonths * 'Distributor Month'[Sellout Qty Reported]) * 'Distributor Month'[Sell In Unit Price])</code>
Sell-in minus Sell-out Units	Units shipped into the channel but not sold out in the period.	<code>[Sell-in Units] - [Sell-out Units]</code>

Demand Forecast

Measure	Definition	DAX
Forecast Units	MRP forecast units.	<code>SUM('Demand Forecast'[Forecast Qty])</code>
Forecast Bias %	(Forecast - actual) / actual. Positive = over-forecast (ageing stock risk), negative = under-forecast (stockout risk).	<code>DIVIDE([Forecast Units] - [Sales Volume], [Sales Volume])</code>
Forecast Error % (WAPE)	Weighted absolute percentage error at SKU x month (sum of	<code>forecast - actual</code>

Production

Measure	Definition	DAX
Planned Units	Planned production units.	<code>SUM('Production'[Planned Qty])</code>
Actual Units	Units produced (good + scrap).	<code>SUM('Production'[Actual Qty])</code>
Good Units	Units passing quality.	<code>SUM('Production'[Good Qty])</code>
Scrap Units	Units scrapped.	<code>SUM('Production'[Scrap Qty])</code>
Production Plan Attainment %	Actual / planned units - the figure Operations reports.	<code>DIVIDE([Actual Units], [Planned Units])</code>
Good Output Attainment %	Good / planned units - what customers can actually be served from.	<code>DIVIDE([Good Units], [Planned Units])</code>
Scrap Rate %	Scrap / actual units.	<code>DIVIDE([Scrap Units], [Actual Units])</code>

Measure	Definition	DAX
Downtime Hours	Unplanned downtime hours.	<code>DIVIDE(SUM('Production'[Downtime Minutes Amount]), 60)</code>
Downtime Minutes per Order	Average downtime per production order.	<code>DIVIDE(SUM('Production'[Downtime Minutes Amount]), COUNTROWS('Production'))</code>
Scrap Cost	Scrapped units at standard cost; EGP.	<code>SUM('Production'[Scrap Value Amount])</code>
Benchmark Scrap Rate %	Conservative internal benchmark: Plant 1 average scrap rate (all shifts) in the same period. Stretch benchmark = Plant 1 day shifts (~1.8%).	<code>CALCULATE([Scrap Rate %], REMOVEFILTERS('Production Unit'), 'Production Unit'[Plant] = "Plant 1")</code>
Avoidable Scrap Cost	Scrap above the Plant 1 average rate, at standard cost - cost removable by bringing Plant 2 and night shifts to Plant 1 practice (Leak 2).	<code>VAR _benchmark = [Benchmark Scrap Rate %] RETURN SUMX('Production', MAX(0, 'Production'[Scrap Qty] - 'Production'[Actual Qty] * _benchmark) * DIVIDE('Production'[Scrap Value Amount], 'Production'[Scrap Qty]))</code>

Labour

Measure	Definition	DAX
Labour Cost	Base + overtime + allowances; EGP. Restricted (OLS).	<code>SUM('Labour'[Total Labour Cost Amount])</code>
Overtime Cost	Overtime pay; EGP. Restricted (OLS).	<code>SUM('Labour'[Overtime Cost Amount])</code>
Overtime % of Labour Cost	Overtime share of labour cost.	<code>DIVIDE([Overtime Cost], [Labour Cost])</code>
Overtime Hours Total	Overtime hours worked.	<code>SUM('Labour'[Overtime Hours])</code>
Employee Months	Sum of monthly headcount (employee-months).	<code>SUM('Labour'[Headcount])</code>
Average Headcount	Average monthly headcount.	<code>AVERAGEX(VALUES('Date'[Year Month]), CALCULATE(SUM('Labour'[Headcount])))</code>
Overtime Hours per Employee	Overtime hours per employee per month.	<code>DIVIDE([Overtime Hours Total], [Employee Months])</code>
Absence Rate %	Absence hours / scheduled hours.	<code>VAR _abs = SUM('Labour'[Absence Hours]) RETURN DIVIDE(_abs, SUM('Labour'[Attendance Hours]) + _abs)</code>
Labour Cost per Good Unit	Labour cost / good units produced. CAVEAT: payroll appears to be on a larger scale than production volume; use for relative comparison.	<code>DIVIDE([Labour Cost], [Good Units])</code>

CRM Opportunity

Measure	Definition	DAX
Open Pipeline Value	Value of open opportunities.	<code>CALCULATE(SUM('CRM Opportunity'[Opportunity Value]), 'CRM Opportunity'[Is Open] = TRUE())</code>

Measure	Definition	DAX
Stale Pipeline %	Open pipeline already past its expected close date.	<code>DIVIDE(CALCULATE(SUM('CRM Opportunity'[Opportunity Value]), 'CRM Opportunity'[Is Stale] = TRUE()), [Open Pipeline Value])</code>
Win Rate %	Won / closed opportunities (count).	<code>DIVIDE(CALCULATE(COUNTROWS('CRM Opportunity'), 'CRM Opportunity'[Is Won] = TRUE()), CALCULATE(COUNTROWS('CRM Opportunity'), 'CRM Opportunity'[Is Open] = FALSE()))</code>
Closed Won Value	Value of won opportunities (CRM scale differs from ERP revenue).	<code>CALCULATE(SUM('CRM Opportunity'[Opportunity Value]), 'CRM Opportunity'[Is Won] = TRUE())</code>

CRM Activity

Measure	Definition	DAX
Activities	Sales activities logged.	<code>COUNTROWS('CRM Activity')</code>
Activity Hours	Hours of logged sales activity.	<code>DIVIDE(SUM('CRM Activity'[Duration Minutes]), 60)</code>

ML Demand Forecast

Measure	Definition	DAX
ML Forecast Units	Machine-learning forecast units.	<code>SUM('ML Demand Forecast'[ML Forecast Qty])</code>
ML Backtest WAPE %	Out-of-sample weighted absolute % error of the ML model.	<code>VAR _bt = FILTER('ML Demand Forecast', 'ML Demand Forecast'[Record Type] = "backtest") RETURN DIVIDE(SUMX(_bt, ABS('ML Demand Forecast'[ML Forecast Qty] - 'ML Demand Forecast'[Backtest Actual Qty])), SUMX(_bt, 'ML Demand Forecast'[Backtest Actual Qty]))</code>
MRP Backtest WAPE %	Weighted absolute % error of the incumbent MRP forecast over the same backtest months.	<code>VAR _bt = FILTER('ML Demand Forecast', 'ML Demand Forecast'[Record Type] = "backtest") RETURN DIVIDE(SUMX(_bt, ABS('ML Demand Forecast'[MRP Forecast Qty] - 'ML Demand Forecast'[Backtest Actual Qty])), SUMX(_bt, 'ML Demand Forecast'[Backtest Actual Qty]))</code>
Rule Backtest WAPE %	Error of the rule-based challenger (MRP x trailing 3-month actual/forecast ratio) over the same backtest months.	<code>VAR _bt = FILTER('ML Demand Forecast', 'ML Demand Forecast'[Record Type] = "backtest") RETURN DIVIDE(SUMX(_bt, ABS('ML Demand Forecast'[Rule Forecast Qty] - 'ML Demand Forecast'[Backtest Actual Qty])), SUMX(_bt, 'ML Demand Forecast'[Backtest Actual Qty]))</code>
Rule Forecast Units	Bias-corrected MRP forecast units (rule).	<code>SUM('ML Demand Forecast'[Rule Forecast Qty])</code>

Stock Rebalancing

Measure	Definition	DAX
Excess Stock Value	Stock above target cover, at standard cost.	<code>CALCULATE(SUM('Stock Rebalancing'[Stock Gap Value]), 'Stock Rebalancing'[Stock Gap] > 0)</code>
Stock Shortfall Units	Units below target cover (stockout risk).	<code>-CALCULATE(SUM('Stock Rebalancing'[Stock Gap]), 'Stock Rebalancing'[Stock Gap] < 0) + 0</code>

Data Quality

Measure	Definition	DAX
DQ Violations	Rows violating data-quality rules at the last run.	SUM('Data Quality'[Violations Count])
DQ Rules Breached	Rules with at least one violation.	CALCULATE(COUNTROWS('Data Quality'), 'Data Quality'[Violations Count] > 0)

3. Security and production readiness

3.1 Protecting payroll and commercially sensitive data

Control	Implemented in the prototype	Production hardening
Data minimisation	Employee-level payroll stays in <code>lh_silver</code> ; gold only holds plant × line × shift × month aggregates, with an assertion that no cell has fewer than 5 employees (prevents re-identifying one person's pay).	Keep. Add a separate HR workspace for silver HR tables.
Layer isolation	Three lakehouses. Bronze/silver are engineering-only; business users never get lakehouse or SQL endpoint access, only Build/Read on the semantic model.	Separate <i>Data Engineering</i> and <i>BI</i> workspaces; OneLake data access roles on bronze/silver folders; sensitivity labels (Confidential – HR) on HR tables and the model.
Row-level security	Dynamic RLS role <i>Regional Manager: Region[Region Key] IN <regions mapped to USERPRINCIPALNAME()></i> , plus matching filters on Customer, Distributor and Warehouse. <i>Executive</i> role has no filter.	Mapping table fed from Entra ID security groups (one group per region); role membership assigned to groups, never individuals; test with "View as role" in every release.
Object-level security	Labour cost columns (<code>Total Labour Cost</code> , <code>Overtime Cost</code> , <code>Base Salary</code> , <code>Allowances</code>) are none for Regional Managers — they see hours, absence and scrap, not pay.	Extend OLS to margin/COGS for any external (distributor-facing) audience.
Commercial data	Cost, margin and promotion ROI live only in the governed model; the report is shared, the lakehouse is not.	Disable "export underlying data" for non-executives; Purview DLP policies on the model; audit logs to Log Analytics.
Direct Lake identity	Uses the viewer's identity (SSO) against <code>lh_gold</code> .	Switch to a fixed-identity cloud connection (service principal) so viewers need no lakehouse permission at all; RLS still applies in the model.
AI	ML trains on aggregated sales only; no personal data. Conversational analytics runs through the semantic model, so RLS/OLS apply to every AI answer.	Data agent limited to the certified model; prompts/answers logged; Copilot tenant settings scoped to a security group.

3.2 From prototype to production

Area	What exists now	What is needed before enterprise rollout
Environments	DairyCo_v2_Dev and DairyCo_v2_Prod workspaces on one F2 capacity.	Dev / Test / Prod workspaces; Prod on its own capacity (F64 recommended: Copilot, no Pro licences for viewers, headroom).
Source control	All code in GitHub: notebooks, pipeline JSON, ingestion config, TMDL model, report definition, deploy scripts. Dev workspace connected to Git.	Branch policy (PR + review), feature workspaces per developer, Git as the single source of truth.
Deployment	Fabric deployment pipeline Dev → Prod; scripts deploy notebooks/model/report by name, resolving IDs in the target workspace.	Automated release (GitHub Actions + <code>fabric-cicd</code> or deployment-pipeline API) triggered on merge to <code>main</code> ; deployment rules rebinding the model's Direct Lake source and notebook default lakehouses per stage.
Parameterisation	Notebooks reference lakehouses by <i>name</i> (workspace-relative), pipeline parameters (<code>load_date_filter</code> , <code>full_reload</code>), <code>batch_id</code> injected from the pipeline run.	Fabric Variable Library per stage (source paths, thresholds such as near-expiry 25%, benchmark scrap rate, target cover).

Area	What exists now	What is needed before enterprise rollout
Incremental & replay	Watermark + MERGE for Sales; landing folders by load date; full_reload switch rebuilds from bronze.	Extend incremental to returns/inventory/sell-out; partition fact_sales by month; retention (VACUUM, 30-day time travel).
Monitoring & failure handling	Activity retries (2 × 120 s) and timeouts; bronze logs every file (SUCCEEDED/FAILED) and continues past a bad file; silver hard-stops on value-breaking DQ rules so gold is never refreshed with wrong numbers.	Pipeline failure alert via Fabric Activator / Teams; Capacity Metrics app monitoring; run log table surfaced in an ops report; on-call runbook.
Data-quality monitoring	17 rules, dq_issues (current) and dq_run_history (trend), surfaced on the Operations page.	Thresholds and owners per rule (e.g. alert when duplicate customers > 0), DQ trend in the ops report, data-contract checks on source schema drift.
Performance	V-Order on gold, integer keys, star schema, hidden technical columns, discourageImplicitMeasures.	Monitor Direct Lake fallback/guardrails, Best Practice Analyzer in CI, DAX performance review of heavy measures (WAPE, avoidable scrap).
Lineage & catalogue	Fabric lineage view shows landing → bronze → silver → gold → model → report; every model object has a description.	Endorse (certify) the model; register in Purview / OneLake catalog; business glossary for KPI definitions.
Operating ownership	Owner per source recorded in the ingestion config (Finance/Commercial IT, Sales Ops, Channel Sales, Supply Chain, HR).	RACI: data owners fix source DQ issues; BI team owns model and measures; a KPI council signs off definitions (ends "different versions of the same KPI").

3.3 Data limitations that need a business decision (not a technical fix)

1. **Targets** are about 24× actual revenue, with a fixed implied price of 35 EGP. Finance needs to restate them. Until then attainment is shown only as a relative index.
2. **Stockout rate** cannot be measured. There are no orders, backorders or lost sales, and the stock status is always "Available". The report shows a proxy (share of zero-stock SKU-warehouse-weeks, 0.26%). The real KPI needs order lines with requested vs fulfilled quantity and daily or outlet-level stock.
3. **Cross-source scale.** Warehouse stock, payroll and CRM values are on different scales from invoice sales. This makes absolute ratios such as labour cost per unit, days of cover and CRM value vs revenue unreliable. Relative comparisons and trends still hold. Source owners need to confirm the units.
4. **Distributor feeds.** The stock roll-forward does not balance in 99% of rows, and reported sell-out matches outlet sell-out in only 2% of cells. The distributor data contract should require a roll-forward that reconciles.
5. **Growth periods.** With 20 months of history, only Jan–Aug year-on-year comparisons are like-for-like.

4. AI and advanced analytics

AI earns its place where a decision is repeated often, the pattern cannot be captured in a stable rule, and a better prediction changes the action taken. Where a rule is enough (for example "alert when distributor cover > 1 month"), plain BI or rules are cheaper, easier to explain and should be used instead.

4.1 Use case A (prototyped): SKU demand forecasting → production and stock rebalancing

Business problem	The MRP forecast is biased by category: Milk is under-forecast by about 8% and Yogurt over-forecast by about 19%. The production plan copies the forecast almost 1:1 ($r = 0.99999$), so forecast error goes straight into stock-outs (Milk) and ageing stock (Yogurt, Juice, Cheese). This is the mechanism behind "stockouts while holding excess inventory".
Users	Demand planner (monthly S&OP), plant schedulers, DC managers.
Why AI, not BI/ rules	Demand shifts by SKU (five Milk SKUs doubled while the rest fell 20–27%). A static rule like "last year + x%" cannot track that. A pooled ML model learns trend, seasonality and momentum across all SKUs at once and adapts each month. BI still does the <i>monitoring</i> (bias and WAPE measures).
Data	ERP sales by SKU × month (20 months), the MRP forecast (benchmark), product master (category, shelf life), and weekly warehouse stock for rebalancing. Sufficient for a prototype, not for production. It has one full seasonal cycle and no price, promotion calendar, distribution or holiday (Ramadan) drivers. Production would need 3+ years of history, the promo calendar, outlet sell-out and ideally weekly grain.
Approach	A <i>correction layer</i> on the MRP forecast: <code>HistGradientBoostingRegressor</code> predicts the ratio $\text{actual} \div \text{MRP forecast}$ from the SKU's trailing forecast bias (3-month mean and lags 1–3), 3-month momentum, month of year and category, and the forecast is $\text{MRP} \times \text{predicted ratio}$. Rolling-origin backtest: for each of the last 6 months, train only on earlier months, predict that month and compare against the MRP forecast, a naive "same as last month" forecast and a simple rule ($\text{MRP} \times \text{trailing actual/forecast ratio}$). Everything is logged to the MLflow experiment <code>dairyco-demand-forecast</code> , and the final model is registered as <code>dairyco-demand-forecast-hgb</code> .
Embedding in the process	The pipeline scores next month after every load and writes to <code>lh_gold.ml_demand_forecast</code> . The planner sees ML vs MRP side by side (Operations page) and overrides with reasons, which become training signal. <code>ml_stock_rebalancing</code> gives each warehouse × SKU a <i>Replenish / Hold / Reduce-redeploy</i> action against a target cover of 50% of shelf life.
Value & success measures	Lower forecast WAPE and bias (target: halve the Yogurt over-forecast), lower near-expiry stock value and expired returns, fewer zero-stock SKU-weeks for Milk, and more production runs matched to demand. Measure monthly against the MRP forecast as a champion/challenger.
Risks & governance	Short history (overfitting risk, mitigated by the out-of-sample backtest); the model does not know about promotions or new listings (planner override required); drift (monthly retrain with the backtest gate: promote only if WAPE beats the current model). The data contains no personal information. The model registry holds versioning and lineage, and there is a human in the loop for every production decision.

4.2 Use case B (designed): early-warning channel-risk score for distributors

Business problem	Nile Distribution (301) tripled its stock and lost about 30% of sell-out while ERP showed it <i>growing</i> +5%. By the time sell-in drops, the revenue is already pulled forward and the stock is ageing in the channel.
Users	Channel sales manager and the Commercial Director (monthly distributor review). Credit control as a secondary user.
Why AI	The obvious case can be caught with a rule ($\text{sell-through} < 85\%$ for 2 months), and that rule is shipped now as the <i>Sell-through %</i> and <i>Distributor Months of Cover</i> measures. A model adds value in scoring <i>emerging</i> risk from several weak signals together: cover trend, sell-through, outlet count, return rate, SKU-mix drift and payment behaviour. It also ranks distributors by expected write-back or return exposure.

Data	The distributor platform (sell-out, stock, outlets), ERP sell-in and returns. Partly sufficient. With only 6 distributors there is too little history to train a supervised model. Start with an unsupervised anomaly score (isolation forest or robust z-scores on cover and sell-through) and graduate to supervised once outcomes (write-offs, credit notes) are recorded. Also needs AR/payment data.
Embedding	Monthly score per distributor with its top drivers on the Commercial page, plus an Activator alert when the score crosses a threshold. The rep must log an action (stock audit, reduced allocation, promotion to pull through).
Value & measures	Excess channel stock value, distributor cover back to peer level (about 0.6 months), fewer expired returns from distributors, and forecast quality of sell-in.
Risks	False positives damage relationships, so the score triggers a <i>review</i> , not a sanction. Explainability is mandatory (show drivers). Distributors are external parties, so their data sharing and use must be covered in the distributor agreement.

4.3 Conversational analytics over trusted data

Proposed design: a Fabric Data Agent (or Copilot in Power BI) grounded on the certified DairyCo Analytics semantic model, not on raw tables.

- **Grounding.** The agent generates DAX against the semantic model, so every answer uses the same measures as the dashboards: *Net Revenue*, *Gross Margin %*, *Revenue Growth %* (like-for-like), *Stockout Proxy %*, and so on. All 95 measures and every table and column carry business descriptions (built into the TMDL), which is what the agent reads to choose fields. Add agent instructions (e.g. "revenue means Net Revenue before returns; growth is like-for-like Jan–Aug") and **verified answers** for the 20 most common executive questions.
- **Security.** Queries execute as the signed-in user, so RLS (regional managers see their regions) and OLS (no labour cost for regional users) apply automatically. The agent cannot see anything the user cannot see in the report. It has no access to bronze or silver, so there is no path to employee-level payroll.
- **Semantic and business definitions.** A KPI glossary lives in the model descriptions and a governed "KPI definitions" page. Deliberately unreliable measures say so in their description (*Target Attainment %*: "UNRELIABLE AS ABSOLUTE"; *Stockout Proxy %*: "PROXY"), so the agent repeats the caveat rather than presenting a made-up number.
- **Reducing unsupported or misleading answers:** 1. Restrict the agent to the certified model and a curated perspective (no technical or DQ columns). 2. Always show the generated query and the measure used ("Answer based on *Revenue Growth %*, Jan–Aug 2026 vs 2025"). 3. Instruct it to refuse questions the data cannot answer (e.g. true stockout rate, cost-to-serve) and to name the missing data. 4. Keep an evaluation set of 50 questions with known answers, re-run after every model change, and ship only if accuracy $\geq 95\%$. 5. Log prompts and answers, and let users flag bad answers, which feed the verified-answers list.

4.4 Prototype results

These are the results of a 6-month rolling-origin backtest (Mar–Aug 2026, 24 SKUs, 144 SKU-months, out of sample). WAPE is the weighted absolute percentage error.

Category	MRP forecast (today)	Rule: MRP * trailing 3-month actual/forecast	ML correction model
Yogurt	18.7%	5.2%	4.8%
Milk	8.7%	6.0%	6.4%
Cheese	5.2%	6.7%	5.5%
Juice	4.9%	5.8%	5.7%

Category	MRP forecast (today)	Rule: MRP * trailing 3-month actual/forecast	ML correction model
All SKUs	9.9%	5.9%	5.8%

A naive "same as last month" forecast scores 11.1%. ML bias is -0.01%, against +0.51% overall for MRP. The overall MRP figure hides +19% on Yogurt and -8% on Milk.

Interpretation. 1. The incumbent forecast's error is a stable, SKU-specific bias, not noise. Correcting it cuts error by about 40% overall and by about 75% on Yogurt. 2. **A simple rule gets almost all of the benefit.** The ML model is only marginally better overall and is best on Yogurt. This is exactly the case where BI and rules are the right first step. 3. Recommendation: deploy the rule into the S&OP process now. Keep the ML model as a tracked challenger in MLflow, and promote it once it has features a rule cannot use (promotion calendar, holidays such as Ramadan, weather, outlet sell-out, new listings). 4. Honest limit: an earlier ML version trained on demand *levels* was **worse** than MRP (13.0% WAPE) and was rejected in the backtest. That is exactly the value of the champion/challenger gate.

Rebalancing output (ml_stock_rebalancing). Against a target cover of 50% of shelf life, 141 of 144 DC × SKU cells are flagged *Reduce / redeploy*, with about 1.5M EGP of stock above target. Given the scale caveat on warehouse quantities, the table is meant to prioritise *where* to act (Yogurt and Cheese DCs first, Milk last), not to report an absolute value.

Where it lives in Fabric. - Notebook: nb_05_ml_demand_forecast. - MLflow experiment: dairyco-demand-forecast, with parameters, metrics per model and category, and the model artefact. - Registered model: dairyco-demand-forecast-hgb. - Outputs: lh_gold.ml_demand_forecast and lh_gold.ml_stock_rebalancing. - Report: surfaced on the Operations page (*ML/ MRP/Rule Backtest WAPE %*).

5. Explainer briefs and likely reviewer questions

5.1 The architecture in plain English (60 seconds)

Five source systems drop files into a **landing zone**. A single **configuration file** lists every file, its format, owner and sensitivity, and one notebook ingests them all into **bronze** exactly as received, stamped with where and when they came from. **Silver** turns the raw data into clean, typed, consistent tables: one list of regions, one master customer, CRM and distributor IDs matched to ERP. Every data problem found is written to a **data-quality log** rather than hidden. **Gold** reshapes silver into a **star schema** (facts such as sales, stock and production, surrounded by shared dimensions such as date, product and region). The **semantic model** reads gold directly (**Direct Lake**) and holds every KPI definition once, with security. The **report**, the **AI forecast** and any future **Copilot / data agent** all use that one model, which is what ends "different functions reporting different versions of the same KPI". A **pipeline** runs the chain daily, and **Git plus Dev/Prod workspaces** control change.

5.2 The AI prototype in plain English (60 seconds)

The company's planning system forecasts demand for each product every month, and the factories produce exactly that forecast. It consistently forecasts too little Milk and too much Yogurt, so Milk runs short while Yogurt ages in the warehouse. We trained a machine-learning model on 20 months of sales. It looks at each product's recent months, its trend and the time of year, and learns patterns across all 24 products together. To test it honestly we **pretended to be in the past**: for each of the last six months we trained only on the data available before that month, predicted it, and compared the prediction with what actually sold, alongside the planning system's forecast for the same month. Both the model and a simple bias-correcting rule cut the error by about 40%, and the rule gets almost all of the benefit. So the recommendation is to start with the rule and keep the model as a challenger until richer data (promotion calendar, holidays, outlet sell-out) gives it something a rule cannot capture (numbers in section 4.4). The model's forecast then sets a **target stock level** for each product in each warehouse (half its shelf life of cover), and each line is flagged *replenish*, *hold* or *reduce*. A planner stays in charge and can override it.

5.3 Likely questions and answers

Why a lakehouse and notebooks rather than a warehouse? The sources include JSON API feeds and need MERGE, regex mapping and DQ logging, all of which Spark handles in one engine. If DairyCo's team is SQL-first, gold can move to a Fabric Warehouse without changing the semantic model or report. The trade-off is documented in section 1.2.

Why three lakehouses instead of schemas? Security boundaries. Raw payroll only ever exists in bronze and silver, and business users never get access to those. With one lakehouse, everyone with access would share one permission surface.

How is it incremental? Landing folders are dated. Bronze ingests only files not already logged. Silver's Sales table uses a watermark on ingestion time plus a Delta MERGE on InvoiceID. We proved it by loading up to July and then dropping an August file: 3,723 new lines merged, the watermark advanced, and a re-run changed nothing.

What happens when a load fails? Each activity retries twice. A bad file is logged as FAILED and the other files continue. If a *value-breaking* rule fails in silver (net $\neq$ gross - discount, actual $\neq$ good + scrap), the pipeline stops before gold, so executives never see wrong numbers. Production would add an Activator/Teams alert on failure.

Why Direct Lake? Gold is already Delta/Parquet in OneLake, so the model reads it in place. There is no import refresh, no second copy of the data, and new data appears as soon as gold is written. Import would be the right choice for non-Fabric sources or heavy calculated columns.

How does RLS work, and how do you protect payroll? There is one *Regional Manager* role. A mapping table links each user's login to their regions, and the role filters the Region dimension, which filters every regional fact. Payroll is protected in three layers: employee-level pay never reaches gold (aggregated to plant/line/shift with at least 5 people per cell); labour cost columns are hidden from regional managers by object-level security; and bronze/silver are engineering-only.

Why is target attainment shown as an index? Targets are about 24× actual revenue, and every target implies exactly 35 EGP per unit. That is a units or definition problem in the target file. The brief says not to create unsupported metrics, so the absolute percentage is flagged and the index (region vs company average) is used until Finance restates the targets.

Why is stockout rate a proxy? Measuring stockouts needs demand that could not be served: order lines with requested vs delivered quantity, or outlet out-of-stock checks. None exist, and the stock status field is always "Available". We show the share of warehouse-SKU-weeks with zero available stock (0.26%) and name the data needed.

How did the numbers change from the first presentation? They were recomputed from the governed gold layer. The story is the same, with more depth and firmer evidence. Leak 5 moved from "promotions cannot be measured" to "promotions *can* be measured and they do not pay back", because the promotion-to-invoice link was rebuilt by product, region and date.

How would you take this to production? See section 3.2: Dev/Test/Prod on dedicated capacity, Git with pull-request review, automated deployment with stage rules, a Variable Library for thresholds, Activator alerts, DQ thresholds with named owners, a certified model in the OneLake catalog, and a KPI council that owns the definitions.

What would you do next with more time? 1. A reconciled distributor data contract (roll-forward must balance). 2. Order-line data to build a true fill-rate and stockout KPI. 3. A promotion calendar feature and weekly grain for the forecast. 4. A cost-to-serve model by channel. 5. A data agent with verified answers for the executive team.